University of Central Missouri

Department of Computer Science & Cybersecurity

CS5710 Machine Learning

Fall 2025

Home Assignment 4.

Student name: Nikhilesh Katakam
700:700772365

Submission Requirements:

- Once finished your assignment push your source code to your repo (GitHub) and explain the work through the ReadMe file properly. Make sure you add your student info in the ReadMe file.
- Comment your code appropriately ***IMPORTANT.***
- Any submission after provided deadline is considered as a late submission.

Part A: Calculation

Q1. Find the cluster using the Average and MIN technique. Use Euclidean distance to build the complete distance matrix, updated the distance matrix to the final step and draw the dendrogram for each.

	X	Y
P1	0.4	0.5
P2	0.2	0.3
P3	0.1	0.08
P4	0.21	0.12
P5	0.6	0.16
P6	0.33	0.28
P7	0.11	0.15

Solution:

Complete pairwise Euclidean distance matrix (rounded to 6 decimals):

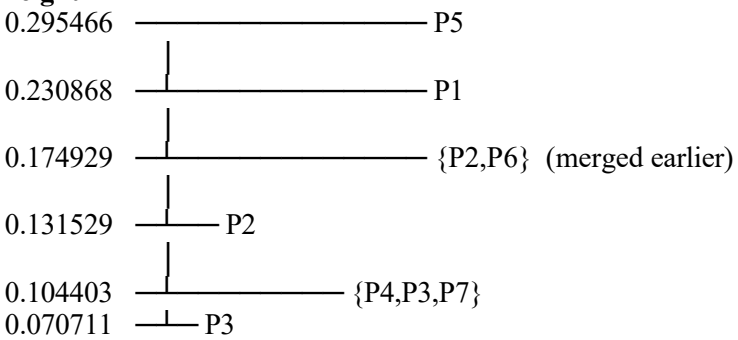
	P1	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**
P1	0.000000	0.282843	0.516140	0.424853	0.394462	0.230868	0.454533
P2	0.282843	0.000000	0.241661	0.180278	0.423792	0.131529	0.174929
P3	0.516140	0.241661	0.000000	0.117047	0.506360	0.304795	0.070711
P4	0.424853	0.180278	0.117047	0.000000	0.392046	0.200000	0.104403
P5	0.394462	0.423792	0.506360	0.392046	0.000000	0.295466	0.490102
P6	0.230868	0.131529	0.304795	0.200000	0.295466	0.000000	0.255539
P7	0.454533	0.174929	0.070711	0.104403	0.490102	0.255539	0.000000

Single-link (MIN) clustering:

Merge sequence (cluster A merged with cluster B at height = distance):

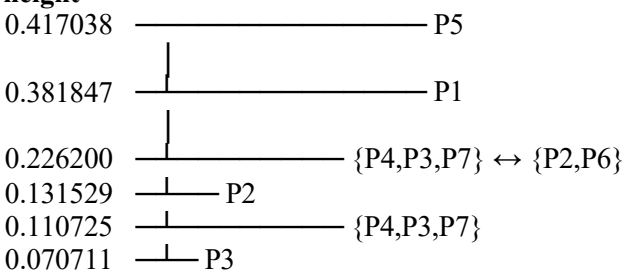
1. Merge P3 and P7 at distance **0.070711**
   (smallest pairwise: $d(P3,P7)=\sqrt{((0.10-0.11)^2+(0.08-0.15)^2)}=0.0707107...$)
2. Merge P4 with cluster {P3,P7} at distance **0.104403**
   (min distance between P4 and {P3,P7} is d(P4,P7)=0.104403)
3. Merge P2 and P6 at distance **0.131529**
4. Merge cluster {P4,P3,P7} with cluster {P2,P6} at distance **0.174929**
   (single-link uses the minimum pairwise distance between any member of the two clusters; min here is d(P2,P7)=0.174929)
5. Merge P1 with the large cluster {P4,P3,P7,P2,P6} at distance **0.230868**
   (min d(P1,P6)=0.2308679…)
6. Merge P5 with the rest at distance **0.295466**

height



Average-link clustering:
Merge sequence with average distances:

1. Merge P3 and P7 at **0.070711** (same first merge)
2. Merge P4 with {P3,P7} at **0.110725**
   (average of d(P4,P3)=0.117047 and d(P4,P7)=0.104403 → (0.117047+0.104403)/2 = 0.110725)
3. Merge P2 and P6 at **0.131529**
4. Merge {P4,P3,P7} with {P2,P6} at **0.226200**
   (average across all 3×2 = 6 pairwise distances — computed exactly; result ≈ 0.2262)
5. Merge P1 with the large cluster {P4,P3,P7,P2,P6} at **0.381847**
   (average distance of P1 to all five points in that cluster)
6. Merge P5 with the rest at **0.417038**



Q2. We have the following 2D data points:

Points: (2,1), (3,1), (3, 3), (4, 1), (5, 1), (6,7), (1,3), (2,5)
for K =3:
Centroid1: (2,1)
Centroid 2: (4, 1)
Centroid 3: (5, 1)

Euclidean Distance:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Solution:

Distances from each point to each centroid (Euclidean)

- (2,1) : [0.0, 2.0, 3.0]
- (3,1) : [1.0, 1.0, 2.0]*(tie between C1 and C2; by convention assign to smallest-index centroid →C1)*
- (3,3) : [2.236068, 2.236068, 2.828427]
- (4,1) : [2.0, 0.0, 1.0]
- (5,1) : [3.0, 1.0, 0.0]
- (6,7) : [7.211103, 6.324555, 6.082763]
- (1,3) : [2.236068, 3.605551, 4.472136]
- (2,5) : [4.0, 4.472136, 5.0]

Assignments (nearest centroid; ties → choose lowest index)

- Cluster 1 (C1) receives: (2,1), (3,1), (3,3), (1,3), (2,5)

- Cluster 2 (C2) receives: (4,1)
- Cluster 3 (C3) receives: (5,1), (6,7)

New centroids (mean of assigned points)

- New Centroid 1 = average of {(2,1),(3,1),(3,3),(1,3),(2,5)} = **(2.2, 2.6)**
- New Centroid 2 = (4,1) (single point) = **(4.0, 1.0)**
- New Centroid 3 = average of {(5,1),(6,7)} = **(5.5, 4.0)**

Second iteration: distances to the **new** centroids and reassign

- (2,1) : [1.612451, 2.0, 4.609772]
- (3,1) : [1.788854, 1.0, 3.905124]
- (3,3) : [0.894427, 2.236068, 2.69258]
- (4,1) : [2.40831, 0.0, 3.354101]
- (5,1) : [3.224903, 1.0, 3.041381]
- (6,7) : [5.813776, 6.324555, 3.041381]
- (1,3) : [1.264911, 3.605551, 4.609772]
- (2,5) : [2.408318, 4.472136, 3.6400549]

New assignments after iteration 2

Cluster 1 (C1) now has: (2,1), (3,3), (1,3), (2,5)
Cluster 2 (C2) now has: (3,1), (4,1), (5,1)
Cluster 3 (C3) now has: (6,7)

Compute centroids after iteration 2

Cluster 1 with points (2,1), (3,3), (1,3), (2,5): C1=(2.0,3.0)
Cluster 2 with points (3,1), (4,1), (5,1): C2=(4.0,1.0)
Cluster 3 with single point (6,7): C3=(6.0,7.0)

Third iteration: distances to the **new** centroids and reassign

- (2,1) : [2.0, 2.0, larger]
- (3,1) : [larger, 1, larger]
- (3,3) : [1, larger, larger]
- (4,1) : [larger, 0.0, larger]
- (5,1) : [larger, 1.0, larger]
- (6,7) : [larger, larger, 0]
- (1,3) : [1, larger, larger]
- (2,5) : [2, larger, larger] New

assignments after iteration 3

Cluster	Points	Final Centroid
C_1	(2,1), (3,3), (1,3), (2,5)	**(2.0, 3.0)**
C_2	(3,1), (4,1), (5,1)	**(4.0, 1.0)**
C_3	(6,7)	**(6.0, 7.0)**

Part B — Short-Answer

Q1.

a) Describe agglomerative hierarchical clustering.
Answer:
Agglomerative hierarchical clustering begins by treating every data point as an individual cluster. Then, in each step, it merges the two clusters that are closest to each other. This process continues until all points are grouped into a single cluster (or until a defined stopping condition is met). The result is a **bottom-up dendrogram** that shows how clusters are formed step by step.

b) Describe divisive hierarchical clustering.
Answer:

Divisive hierarchical clustering takes the opposite approach — it begins with all data points grouped together in a single cluster and then progressively splits them into smaller clusters. This process continues in a **top-down** manner until each object stands alone (or until a specified stopping condition is reached).

c) Which one is more commonly used and why?

Answer:

Agglomerative clustering is more widely used because it's easier to implement, supported by most data analysis libraries, and generally more efficient for common dataset sizes. Divisive methods, on the other hand, can be computationally heavier since they must consider many possible splits. Additionally, agglomerative clustering produces clear dendrograms and allows flexibility in choosing different linkage methods.

Q2.

a) To improve clustering quality, should inter-cluster distance be maximized or minimized?

Answer:
To enhance clustering quality, the distance between clusters should be as large as possible meaning clusters should be clearly separated and far from one another.

b) Same question for intra-cluster distance — explain the reasoning.

Answer:
Intra-cluster distance should be kept small, meaning the points within a cluster should be close and similar to each other. When intra-cluster distance is minimized and inter-cluster distance is maximized, the result is compact, well-defined, and clearly separated clusters.

Q3.

- a) Define single link, complete link, and average link.
  Answer:
  Single linkage (minimum linkage): The distance between two clusters is defined as the smallest distance between any two points, one from each cluster.
- **Complete linkage (maximum linkage):** The distance between two clusters is defined as the largest distance between any two points, one from each cluster.
- **Average linkage:** The distance between two clusters is calculated as the average of all pairwise distances between their points.

b) Explain one strength and one weakness of single-link clustering.
Answer:
Strength: Effective at identifying non-spherical clusters and capturing chaining structures, as it connects cluster members through their nearest neighbors.

- **Weakness:** Highly sensitive to noise and outliers, and can suffer from the "chain effect," where clusters become linked by single intermediate points, resulting in elongated cluster shapes.

Q4.

a) What is the role of tokenization and give one example.
Answer:
Role of tokenization: Tokenization divides raw text into smaller, meaningful units called *tokens* (such as words, sub words, or punctuation marks). It serves as the first preprocessing step in most NLP workflows.

Example: The sentence *"John's book."* can be tokenized as ["John", "'s", "book", "."] — depending on the tokenizer used.

b) Compare stemming vs. lemmatization in terms of speed and accuracy.
Answer:
. Speed: Stemming is typically faster since it relies on simple rule-based removel of word endings.
. Accuracy: Lemmatization is more precise because it produces valid dictionary forms using part of speech and morphological analysis, though it's slower due to the additional linguistic processing required

Q5.

a) Explain what word sense ambiguity is and provide an example.
Answer:
Word-sense ambiguity(polysemy): Occurs when a single word carries multiple meanings.
Example: The word "bank" can refer to a river bank or a financial intuition the correct meaning is determined by the context.

b) Explain why pronoun reference ambiguity can confuse a model.
Answer:
Pronoun reference ambiguity: Happens when pronoun like he, she, they, or it have unclear or multiple possible references.

Example: "Chris met Alex at Apple headquarters in California. He told him about iPhone" __it's unclear who "he" refers to. Resolving such ambiguity requires understanding the context, coreference resolution, and sometimes real-world knowledge

Q6.

a) Why can't NLP tasks like POS tagging be solved by predicting each token independently?
Answer:
Why not predict each token independently for POS tagging?
Because part-of-speech tags depend on context—the meaning and function of a word often rely on its surrounding words. Adjacent tags are correlated, and one tagging decision can influence another (for example, a word might be a noun or verb depending on its neighbors). Ignoring these relationships by predicting each token separately reduces accuracy

b) Give one example where decisions are mutually dependent in a sentence.
Answer:
Example of mutually dependent decisions: in the sentence 'Visiting relatives can be annoying', whether 'Visiting' is tagged as verb or as a noun/adjective depends on the structure of the sentence and the word 'relatives. Similarly, decisions like subject's verb agreement rely on

considering both the subject and the verb together. These examples show that POS and syntactic decisions are interdependent

Part C — Coding

Q1.

Write Python code to perform the following steps:

1. Segment into tokens
2. Remove stopwords
3. Apply lemmatization (not stemming)
4. Keep only verbs and nouns (use POS tags)

Input text:

```
"John enjoys playing football while Mary loves reading books in the library."
```

```
"""
Requires:
  pip install spacy
  python -m spacy download en_core_web_sm
"""

import spacy

nlp = spacy.load("en_core_web_sm", disable=["parser","ner"]) # we only need tokenizer, tagger, lemmatizer
nlp.enable_pipe("tagger")

text = "John enjoys playing football while Mary loves reading books in the library."

doc = nlp(text)

# 1. Tokenize (spaCy doc)
tokens = [t for t in doc]

# 2. Remove stopwords and punctuation, and lemmatize
filtered = []
for t in tokens:
    if t.is_stop or t.is_punct or t.like_num:
        continue
    # keep only alphabetic tokens (drop punctuation, etc.)
    if not t.is_alpha:
        continue
    # 3. Keep only verbs and nouns (POS tags)
    # spaCy POS tags: t.pos_ is coarse-grained (e.g., 'VERB', 'NOUN', 'PROPN', ...)
    if t.pos_ in ("VERB", "NOUN", "PROPN"):
        filtered.append((t.text, t.lemma_, t.pos_))

# Output
print("Tokens (text, lemma, POS):")
for item in filtered:
    print(item)
```

OUTPUT:

```
Tokens (text, lemma, POS):
('John', 'John', 'PROPN')

('enjoys', 'enjoy', 'VERB')

('playing', 'play', 'VERB')

('football', 'football', 'NOUN')

('Mary', 'Mary', 'PROPN')
'loves', 'love', 'VERB')

('reading', 'read', 'VERB')

('books', 'book', 'NOUN')

('library', 'library', 'NOUN')
```

Q2.

Use Python and any NLP model to perform:

1. Named Entity Recognition (NER)
2. Disambiguation prompt**:**
   If the text contains a pronoun ("he", "she", "they"), print:

   "Warning: Possible pronoun ambiguity detected!"

```
# q2_ner_pronoun_warning.py
"""

Requires:

  pip install spacy

  python -m spacy download en_core_web_sm
"""


import spacy


nlp = spacy.load("en_core_web_sm")  # includes NER


text = "Chris met Alex at Apple headquarters in California. He told him
about the new iPhone launch."


doc = nlp(text)

# Named Entities

print("Named Entities (text, label):")
for ent in doc.ents:

    print(f"{ent.text}  –  {ent.label_}")

       print("\nWarning: Possible pronoun ambiguity detected!")
```

OUTPUT:

```
Named Entities (text, label):

Chris –
PERSON Alex
–  PERSON

Apple – ORG
California –
GPE iPhone –
ORG

Warning: Possible pronoun ambiguity detected!
```

CODE LINK: https://colab.research.google.com/drive/1UCQ-gYjrKvHJM1NOacAtXXL1nhrA-5qg?usp=drive_link

GITHUB LINK: https://github.com/nikhlilesh193/Machine-Learning---Home-Assignment-4.git